บทที่ 4

การจัดการสถานะ พร็อพส์ และแบบฟอร์มรับข้อมูล

อินเทอร์เฟซของโมบายแอปพลิเคชันที่มีการตอบโต้ที่ดีจำเป็นต้องประมวลผลการเปลี่ยนแปลงข้อมูลตามพฤติกรรมของผู้ใช้ เช่น การพิมพ์ข้อความ การสลับหน้าจอปุ่มตัวเลือก หรือการกดส่งฟอร์ม ใน React Native เราใช้แนวคิดของ **State (สถานะ)** ร่วมกับ **Props (คุณสมบัติ)** ในการเป็นตัวควบคุมโครงสร้างข้อมูลและการเรนเดอร์เนื้อหาใหม่ (Re-render) เพื่ออัปเดตหน้าจอให้ทันสมัยโดยอัตโนมัติ

4.1 แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook

1. State คืออะไร?

State คือ ข้อมูลภายในคอมโพเนนต์ (React, 2024) ที่สามารถเปลี่ยนแปลงค่าได้ตลอดเวลา (Mutable) ในแต่ละรอบอายุการทำงาน (Lifecycle) ของคอมโพเนนต์ เมื่อค่าของ State ถูกแก้ไขด้วยฟังก์ชันเฉพาะ React จะทำการวาดคอมโพเนนต์ย่อยชิ้นนั้น ๆ ใหม่บนหน้าจอ (Re-render) ทันที เพื่อสะท้อนการเปลี่ยนแปลงให้ผู้ใช้งานเห็นหน้าจอใหม่

2. การเรียกใช้งาน useState Hook

ใน Functional Components เราประกาศสถานะเริ่มต้นและฟังก์ชันในการเปลี่ยนแปลงค่าสเตตผ่าน `useState` Hook (React, 2024):

```
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function CounterApp() {
 // ประกาศ State ชื่อ count มีค่าเริ่มต้นเป็น 0 และมีฟังก์ชัน setCount ในการแก้ไขค่า
  const [count, setCount] = useState<number>(0);

  return (
    <View style={styles.container}>
      <Text style={styles.text}>จำนวนคลิกปัจจุบัน: {count}</Text>
      {/* เมื่อคลิกปุ่ม ให้เพิ่มค่า count ขึ้นทีละ 1 */}
      <Button title="กดเพิ่มจำนวน" onPress={() => setCount(count + 1)} />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

```
    justifyContent: 'center',
    alignItems: 'center',
  },
  text: {
    fontSize: 20,
    marginBottom: 20,
  },
});
```

4.2 ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props

การทำความเข้าใจข้อแตกต่างระหว่าง State และ Props ถือเป็นรากฐานสำคัญของการจัดสถาปัตยกรรมข้อมูลใน React Native:

ตารางที่ 4.1 การเปรียบเทียบคุณลักษณะระหว่าง State และ Props

คุณลักษณะ	State (สถานะภายใน)	Props (คุณสมบัติภายนอก)
ที่มา (Source)	กำหนดและสร้างขึ้นภายในคอมโพเนนต์ตัวเอง	ส่งผ่านมาจากคอมโพเนนต์แม่ (Parent)
การแก้ไขค่า (Mutability)	เปลี่ยนแปลงได้อิสระโดยเรียกใช้ฟังก์ชัน Setter (เช่น setCount)	เปลี่ยนแปลงค่าไม่ได้จากภายในลูก (ใช้อ่านอย่างเดียว - Read-Only)
ตัวเหนี่ยวนำให้เกิดการวาดหน้าจอใหม่ (Re-render Trigger)	ส่งผลให้เรนเดอร์ใหม่ทันทีเมื่อเรียกฟังก์ชันอัปเดตค่า	ส่งผลให้คอมโพเนนต์ลูกเรนเดอร์ใหม่ทันทีเมื่อแม่เปลี่ยนค่า Props
บทบาทหน้าที่หลัก	เก็บข้อมูลประวัติย่อย ความชอบ หรืออินพุตที่ผู้ใช้กำลังพิมพ์	ส่งผ่านการตั้งค่า หน้าตา หรือสั่งทำเหตุการณ์กลับไปหาแม่

4.3 การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput

ในการพัฒนาเว็บหรือแอปแบบดั้งเดิม ตัวอัปเดตข้อความจะถูกผูกไว้ภายในระบบของแป้นพิมพ์หรือ DOM ดั้งเดิม แต่ใน React Native การพัฒนาแบบฟอร์มที่เป็นระเบียบจะใช้แนวคิด **Controlled Components** (React Native, 2024) ซึ่งหมายถึงการนำ State ของ React มาเป็น "แหล่งข้อมูลที่ถูกต้องเพียงหนึ่งเดียว (Single Source of Truth)" สำหรับคอมโพเนนต์อินพุตนั้น ๆ

1. วิธีการสร้าง Controlled Input

เราจะสร้าง State คอยบันทึกตัวอักษร และผูกพร็อพส์ value เข้ากับสเตตนั้น พร้อมสั่งการผ่านเหตุการณ์ onChangeText ให้นำข้อความสตริงล่าสุดไปอัปเดตสเตตทันที:

```
import { useState } from 'react';
import { View, TextInput, Text, StyleSheet } from 'react-native';

export default function ControlledInput() {
  const [username, setUsername] = useState<string>('');

  return (
    <View style={styles.container}>
   <TextInput
        style={styles.input}
        value={username}
        onChangeText={setUsername} // มีค่าเท่ากับ (text) => setUsername(text)
        placeholder="กรอกชื่อผู้ใช้ของคุณ..."
   />
      <Text style={styles.previewText}>ค่าสเตตปัจจุบัน: {username}</Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#94a3b8',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
  },
  previewText: {
    marginTop: 10,
    color: '#64748b',
  },
});
```

4.4 การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up

ข้อมูลใน React จะไหลเวียนไปในทิศทางเดียวจากบนลงล่าง (Parent to Child) หากเราต้องการให้คอมโพเนนต์ลูกสลับกันเปลี่ยนแปลง หรือส่งข้อมูลอินพุตจากช่องกรอกในคอมโพเนนต์ลูก

ขึ้นไปเปลี่ยนค่าที่คอมโพเนนต์แม่ เราจำเป็นต้องประยุกต์ใช้เทคนิค **Lifting State Up (การยกสถานะขึ้น)**

แนวปฏิบัติของการ Lifting State Up

1. **ยกตำแหน่งเก็บ State:** นำ State ไปเก็บไว้ในคอมโพเนนต์ที่เป็นบรรพบุรุษร่วมที่ใกล้ที่สุด (Common Ancestor)

2. **ส่งผ่านฟังก์ชันอัปเดตค่า:** ตัวคอมโพเนนต์แม่จะสร้างฟังก์ชันการประมวลผล แล้วจัดส่งฟังก์ชันนี้ไปเป็นหนึ่งใน Props ของคอมโพเนนต์ลูก

3. **การโทรกลับ (Callback):** เมื่อลูกสัมผัสหรือพิมพ์รับข้อความ จะทำการกระตุ้นฟังก์ชัน Props นั้น ส่งข้อมูลย้อนกลับขึ้นไปอัปเดตค่าสเตตบนคอมโพเนนต์แม่

```
// 1. ชิ้นส่วนลูก (Child) - ช่องป้อนข้อมูล
interface SearchInputProps {
  query: string;
  onQueryChange: (text: string) => void;
}

function SearchInput({ query, onQueryChange }: SearchInputProps) {
  return (
    <TextInput
      value={query}
      onChangeText={onQueryChange} // กระตุ้น Callback ส่งค่ากลับขึ้นด้านบน
      placeholder="พิมพ์ค้นหาชื่อสินค้า..."
      style={{ borderWidth: 1, padding: 8, borderRadius: 6 }}
  />
 );
}

// 2. ชิ้นส่วนแม่ (Parent) - เก็บสเตตหลัก
import { useState } from 'react';
import { View, Text } from 'react-native';

export default function ProductCatalog() {
  const [searchQuery, setSearchQuery] = useState<string>('');

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 18, marginBottom: 8 }}>แคตตาล็อกสินค้า</Text>
      {/* ส่งผ่านทั้งค่าสเตตและฟังก์ชันอัปเดตสเตตไปให้คอมโพเนนต์ลูก */}
      <SearchInput query={searchQuery} onQueryChange={setSearchQuery} />
      <Text style={{ marginTop: 10 }}>คุณกำลังค้นหาคำว่า: {searchQuery}</Text>
  </View>
 );
}
```

4.5 การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form)

นอกจากการพิมพ์เก็บข้อความธรรมดาแล้ว แบบฟอร์มในโมบายแอปพลิเคชันยังนิยมใช้ในการประมวลผลคำนวณทางคณิตศาสตร์จากตัวเลขอินพุตที่ผู้ใช้พิมพ์เข้ามา เช่น ยอดราคาสินค้ารวม หรือดัชนีสุขภาพ โดยข้อมูลอินพุตจาก `TextInput` จะถูกส่งคืนมาเป็นประเภทสตริง (`string`) เสมอ ดังนั้น ผู้พัฒนาจึงต้องแปลงค่าเป็นตัวเลขจริงก่อนเริ่มการคำนวณเพื่อป้องกันข้อผิดพลาดเชิงโปรแกรม (NaN)

1. วิธีการผูกและแปลงค่าตัวเลข (Numeric Binding & Parsing)

• `keyboardType="numeric"`: เปิดแป้นพิมพ์ตัวเลข

• `parseFloat()` / `parseInt()`: ใช้สำหรับแปลงข้อความที่พิมพ์เป็นตัวเลขจำนวนจริงหรือทศนิยม

• **การป้องกันความผิดพลาด (Defensive Programming):** ตรวจสอบว่าผู้ใช้กรอกข้อความว่างเปล่า หรือหารด้วยศูนย์ (Division by Zero) หรือไม่ ก่อนทำรายการคำนวณ

2. โค้ดตัวอย่างคอมโพเนนต์เครื่องคำนวณดัชนีมวลกาย (BMI Calculator)

```
import { useState } from 'react';
import { View, Text, TextInput, Pressable, StyleSheet } from 'react-
native';

export default function BMICalculator() {
  const [weight, setWeight] = useState<string>(''); // เก็บน้ำหนัก (กิโลกรัม) เป็น
ข้อความ
  const [height, setHeight] = useState<string>(''); // เก็บส่วนสูง (เซนติเมตร) เป็น
ข้อความ
  const [bmi, setBmi] = useState<number | null>(null); // เก็บคะแนน BMI คำนวณ
เสร็จ
  const [category, setCategory] = useState<string>(''); // เก็บเกณฑ์สุขภาพ

  const handleCalculate = () => {
    const w = parseFloat(weight);
    const h = parseFloat(height) / 100; // แปลงหน่วยเซนติเมตรเป็นเมตร

    if (w > 0 && h > 0) {
      const calculatedBmi = w / (h * h);
      setBmi(calculatedBmi);

    // แปลผลเกณฑ์ BMI (อ้างอิงตามเกณฑ์เอเชียแปซิฟิก)
      if (calculatedBmi < 18.5) {
```

```
        setCategory('น้ำหนักน้อยกว่าเกณฑ์ (ผอม)');
      } else if (calculatedBmi < 23) {
        setCategory('น้ำหนักปกติ (สุขภาพดี)');
      } else if (calculatedBmi < 25) {
        setCategory('น้ำหนักเกิน (ท้วม)');
      } else {
        setCategory('ภาวะอ้วน (Obese)');
      }
    } else {
      setBmi(null);
      setCategory('กรุณากรอกข้อมูลส่วนสูงและน้ำหนักให้ครบถ้วนถูกต้อง');
    }
  };

  return (
    <View style={styles.card}>
      <Text style={styles.cardTitle}>เครื่องคำนวณหาค่า BMI</Text>

      {/* ช่องกรอกน้ำหนัก */}
      <Text style={styles.label}>น้ำหนักตัว (กิโลกรัม)</Text>
      <TextInput
        style={styles.input}
        placeholder="ตัวอย่าง 65"
        placeholderTextColor="#64748b"
        keyboardType="numeric"
        value={weight}
        onChangeText={setWeight}
      />

      {/* ช่องกรอกส่วนสูง */}
      <Text style={styles.label}>ส่วนสูง (เซนติเมตร)</Text>
      <TextInput
        style={styles.input}
        placeholder="ตัวอย่าง 170"
        placeholderTextColor="#64748b"
        keyboardType="numeric"
        value={height}
        onChangeText={setHeight}
      />

      {/* ปุ่มสั่งคำนวณจัดด้วย Pressable */}
      <Pressable
        onPress={handleCalculate}
        style={({ pressed }) => [
          styles.button,
          pressed ? styles.buttonPressed : null
        ]}
      >
```

```
        <Text style={styles.buttonText}>ประมวลผลดัชนีมวลกาย</Text>
      </Pressable>

      {/* พื้นที่แสดงผลการคำนวณแบบมีเงื่อนไข (Conditional Rendering) */}
      {bmi !== null && (
        <View style={styles.resultBox}>
          <Text style={styles.resultBmiText}>ค่า BMI ของคุณ:
{bmi.toFixed(1)}</Text>
          <Text style={styles.resultCatText}>เกณฑ์ของคุณ: {category}</Text>
        </View>
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 16,
  },
  cardTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 12,
  },
  label: {
    color: '#cbd5e1',
    fontSize: 13,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 44,
    paddingHorizontal: 12,
    color: '#ffffff',
    marginBottom: 12,
  },
  button: {
```

```
    backgroundColor: '#10b981', // สีเขียวนีออนสำหรับคำนวณสุขภาพ
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
  },
  buttonPressed: {
    backgroundColor: '#059669',
  },
  buttonText: {
    color: '#ffffff',
    fontWeight: 'bold',
  },
  resultBox: {
    marginTop: 16,
    padding: 12,
    backgroundColor: '#0f172a',
    borderRadius: 8,
    alignItems: 'center',
  },
  resultBmiText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  resultCatText: {
    color: '#38bdf8',
    fontSize: 14,
    marginTop: 4,
  },
});
```

4.6 การตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการแจ้งเตือนข้อผิดพลาด (Error Feedback)

แอปพลิเคชันคุณภาพสูงต้องช่วยป้องกันไม่ให้ผู้ใช้ส่งข้อมูลที่ผิดรูปแบบไปยังเซิร์ฟเวอร์ โดยระบบควรทำการตรวจสอบข้อผิดพลาด (Validation) ทันทีบนตัวเครื่อง (Client-side) และแสดงคำเตือนอย่างเด่นชัด

1. วิธีตรวจสอบข้อมูลแบบเรียลไทม์ (Real-time Validation)

เราสามารถตรวจสอบค่าของตัวแปรสเตตในแต่ละจังหวะการพิมพ์ และเก็บข้อความผิดพลาดลงในสถานะ `errors` หรือคำนวณจากสถานะปัจจุบันได้โดยตรง:

```
// ตรวจสอบว่ารูปแบบอีเมลถูกต้องหรือไม่โดยใช้ Regular Expression (Regex)
```

```
const validateEmail = (emailText: string) => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(emailText)) {
    return 'รูปแบบอีเมลไม่ถูกต้อง';
  }
  return '';
};
```

2. การจัดการสถานะปุ่มกดสัมผัสและ Error UI

• **กล่องรับข้อความสีส้ม/แดง:** กำหนดสไตล์ขอบช่องพิมพ์ให้แดงขึ้นเมื่อสเตตข้อผิดพลาดไม่มีค่าว่าง

• **การสั่งปิดความสามารถของปุ่มสัมผัส (Disable Button):** นำค่าความถูกต้องของข้อมูล (เช่น `isFormValid`) มาเป็นตัวกำหนดค่าการตอบสนองของ `Pressable` หากฟอร์มยังกรอกไม่สมบูรณ์ ปุ่มจะจางลงและไม่รับคำสั่งนิ้วสัมผัส

4.7 โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์

โค้ดเต็มด้านล่างนี้แสดงวิธีการรวบรวมสถานะหลายช่องรับอินพุต การทำ Validation ข้อมูล และการปรับแต่งปุ่มสัมผัสอย่างลื่นไหลเมื่อเงื่อนไขถูกต้องสมบูรณ์:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  Alert
} from 'react-native';

export default function RegisterForm() {
 // 1. ประกาศตัวแปรสำหรับผูกฟอร์มรับอินพุต
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');

 // 2. ประกาศตัวแปรเก็บสถานะขอบบกพร่อง
  const [emailError, setEmailError] = useState<string>('');
  const [passwordError, setPasswordError] = useState<string>('');

 // 3. ฟังก์ชันดักจับความถูกต้องของอีเมล
  const handleEmailChange = (text: string) => {
    setEmail(text);
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (text.trim() === '') {
      setEmailError('กรุณากรอกอีเมลของคุณ');
```

```
    } else if (!emailRegex.test(text)) {
      setEmailError('อีเมลต้องเป็นรูปแบบที่ถูกต้อง เช่น name@domain.com');
    } else {
      setEmailError('');
    }
  };

  // 4. ฟังก์ชันดักจับความยาวรหัสผ่าน
  const handlePasswordChange = (text: string) => {
    setPassword(text);
    if (text.trim() === '') {
      setPasswordError('กรุณากรอกรหัสผ่าน');
    } else if (text.length < 6) {
      setPasswordError('รหัสผ่านต้องมีความยาวอย่างน้อย 6 ตัวอักษร');
    } else {
      setPasswordError('');
    }
  };

  // 5. ประเมินสัญญานว่าฟอร์มโดยรวมสมบูรณ์พร้อมส่งหรือไม่
  const isFormValid =
    email.trim() !== '' &&
    password.trim() !== '' &&
    emailError === '' &&
    passwordError === '';

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert('ลงทะเบียนสำเร็จ', `อีเมล: ${email}\nรหัสผ่านของคุณปลอดภัยเรียบร้อย`);
    }
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>
        <Text style={styles.title}>สร้างบัญชีสมาชิก</Text>

        {/* ฟอร์มรับอีเมล */}
        <View style={styles.inputGroup}>
          <Text style={styles.label}>ที่อยู่อีเมล</Text>
          <TextInput
            style={[
              styles.input,
              emailError ? styles.inputInvalid : null
      ]}
            placeholder="ตัวอย่าง user@email.com"
            placeholderTextColor="#64748b"
            value={email}
            onChangeText={handleEmailChange}
            keyboardType="email-address"
```

```
            autoCapitalize="none"
      />
        {emailError ? <Text style={styles.errorText}>{emailError}</Text> :
null}
        </View>

        {/* ฟอร์มรับรหัสผ่าน */}
        <View style={styles.inputGroup}>
          <Text style={styles.label}>รหัสผ่าน (Password)</Text>
          <TextInput
            style={[
              styles.input,
              passwordError ? styles.inputInvalid : null
      ]}
            placeholder="ตั้งรหัสผ่านอย่างน้อย 6 ตัว"
            placeholderTextColor="#64748b"
            value={password}
         onChangeText={handlePasswordChange}
            secureTextEntry={true}
            autoCapitalize="none"
      />
          {passwordError ? <Text
style={styles.errorText}>{passwordError}</Text> : null}
        </View>

        {/* ปุ่มบันทึกข้อมูลโต้ตอบตามความถูกต้องของข้อมูล */}
        <Pressable
          disabled={!isFormValid} // ล็อกไม่ให้คลิกปุ่มหากฟอร์มไม่สมบูรณ์
          onPress={handleSubmit}
          style={({ pressed }) => [
            styles.submitButton,
      !isFormValid ? styles.submitButtonDisabled : null,
            pressed && isFormValid ? styles.submitButtonPressed : null
      ]}
        >
          <Text style={styles.submitButtonText}>ลงทะเบียนบัญชีใหม่</Text>
        </Pressable>

      </View>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
  },
```

```
  container: {
    padding: 24,
    marginHorizontal: 16,
    backgroundColor: '#1e293b',
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 24,
  },
  inputGroup: {
    marginBottom: 16,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    marginBottom: 6,
    fontWeight: '500',
  },
  input: {
    height: 48,
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 15,
  },
  inputInvalid: {
    borderColor: '#f43f5e', // เปลี่ยนกรอบเป็นสีส้มแดงเตือนภัย
  },
  errorText: {
    fontSize: 12,
    color: '#f43f5e',
    marginTop: 6,
  },
  submitButton: {
    backgroundColor: '#3b82f6',
    height: 48,
    borderRadius: 8,
    justifyContent: 'center',
```

```
    alignItems: 'center',
    marginTop: 10,
  },
  submitButtonDisabled: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    opacity: 0.5,
  },
  submitButtonPressed: {
    backgroundColor: '#2563eb',
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

สรุปท้ายบทเรียน

แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook: สถานะ (State) คือข้อมูลภายในคอมโพเนนต์ที่เปลี่ยนค่าได้ โดยการอัปเดตสเตตผ่านฟังก์ชันแก้ไขของ useState Hook จะกระตุ้นให้ React ทำการวาดอินเตอร์เฟซบนหน้าจอใหม่ (Re-render) โดยอัตโนมัติ เพื่อนำผลลัพธ์ของข้อมูลล่าสุดไปแสดงผลให้กับผู้ใช้ได้อย่างถูกต้องทันที

ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props: State ทำหน้าที่เก็บและจัดการข้อมูลที่มีการเคลื่อนไหวอยู่ภายในคอมโพเนนต์ชิ้นนั้น ๆ ในขณะที่ Props เป็นคุณสมบัติภายนอกที่ส่งผ่านจากคอมโพเนนต์แม่ลงมายังคอมโพเนนต์ลูกแบบอ่านได้อย่างเดียว (Read-Only) ซึ่งการประสานงานระหว่างสองอย่างนี้จะเหนี่ยวนำให้คอมโพเนนต์ลูกเรนเดอร์ใหม่โดยอัตโนมัติเมื่อข้อมูลมีการเปลี่ยนแปลง

การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput: การจัดการอินพุตฟอร์มใน React Native ยึดหลัก Controlled Components โดยมีสเตตเป็นแหล่งข้อมูลที่ถูกต้องที่สุดเพียงแหล่งเดียว (Single Source of Truth) ซึ่งเป็นการผูกสเตตเข้ากับคุณสมบัติ value ของกล่องรับข้อมูลข้อความ TextInput และกระตุ้นฟังก์ชันเซ็ตเตอร์ผ่านเหตุการณ์ onChangeText เพื่อเก็บตัวอักษรทุกตัวลงในสเตตแบบเรียลไทม์

การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up: เพื่อรองรับทิศทางการไหลเวียนข้อมูลทางเดียวจากแม่ไปลูก (One-way Data Flow) การส่งผ่านข้อมูลย้อนกลับจากลูกขึ้น

ไปหาแม่จะใช้เทคนิค Lifting State Up โดยการย้ายสเตตหลักขึ้นไปไว้บนคอมโพเนนต์บรรพบุรุษร่วม แล้วส่งฟังก์ชันการจัดการ (Callback Function) ผ่าน Props ลงมาให้คอมโพเนนต์ลูกเรียกใช้งานเมื่อต้องการส่งสัญญาณการเปลี่ยนแปลง

การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form): การสร้างฟอร์มสำหรับคำนวณ เช่น เครื่องหาดัชนีมวลกาย (BMI) จะใช้ TextInput ควบคู่ไปกับแป้นพิมพ์ตัวเลข (keyboardType="numeric") และเมื่อสั่งทำงานจะต้องทำการแปลงประเภทข้อมูลจากข้อความสตริงเป็นตัวเลขจำนวนจริงผ่าน parseFloat() ก่อนเริ่มกระบวนการคำนวณ พร้อมทั้งจัดระบบป้องกันการคำนวณที่ผิดพลาดจากข้อมูลที่ว่างเปล่าหรือเป็นศูนย์

การตรวจสอบความถูกต้องของฟอร์มและการแจ้งเตือนข้อผิดพลาด: ระบบแบบฟอร์มที่ดีจำเป็นต้องมี Form Validation เพื่อตรวจสอบความถูกต้องของอีเมลหรือรหัสผ่านทันทีในระบบ Client-side ก่อนนำส่งข้อมูลจริง โดยผู้พัฒนาสามารถเก็บข้อมูลข้อผิดพลาดไว้ในสเตตและนำสัญญานตรวจสอบนั้นมาควบคุมการปิดฟังก์ชันปุ่มกด (disabled) รวมถึงเปลี่ยนดีไซน์สีของกรอบอินพุตให้สะดุดตาเพื่อสร้างการตอบสนองที่เข้าใจง่าย

โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์: ตัวอย่างการประกอบฟอร์มลงทะเบียนแสดงผลความสอดคล้องของการนำ TextInput มารับค่าอีเมลและรหัสผ่าน โดยใช้ฟังก์ชันประเมินความถูกต้องของตัวอักษรเพื่ออัปเดตสเตตความผิดพลาดแบบวินาทีต่อวินาที ซึ่งช่วยควบคุมการทำงานของปุ่ม Pressable และเปิดโอกาสให้ผู้ใช้งานกดยืนยันการทำธุรกรรมได้เฉพาะเมื่อข้อมูลในทุกช่องมีรูปแบบที่สมบูรณ์และปลอดภัย

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับแนวคิดเรื่องสถานะ (State) การทำงานร่วมกับ Props โครงสร้าง Controlled Components ของแบบฟอร์ม การส่งค่าข้อมูลกลับขึ้นด้านบนด้วยหลัก Lifting State Up และการดักจับข้อผิดพลาดอินพุต (Form Validation) ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายวัตถุประสงค์และพฤติกรรมการทำงานหลักของ "State" ใน React Native ได้ถูกต้องและสมบูรณ์ที่สุด?

ก. เป็นสไตล์ชีทภายนอกที่ใช้อ่านข้อมูลจากเครื่องระบบปฏิบัติการปลายทางเท่านั้น

ข. เป็นชุดตัวแปรภายในคอมโพเนนต์ที่สามารถเปลี่ยนค่าข้อมูลได้ โดยระบบจะบังคับทำการวาดโครงสร้างหน้าจอใหม่ (Re-render) อัตโนมัติเมื่อค่านั้นเปลี่ยนไป

ค. เป็นพร็อพส์ภายนอกที่ส่งมาจากคอมโพเนนต์แม่เพื่อสแกนล้างหน่วยความจำ RAM

ง. เป็นฟังก์ชันสำหรับส่งข้อความสั้นหรือแจ้งเตือนแบบพุช (Push Notification) ไปยังมือถือเครื่องอื่น

ข้อ 2: เมื่อต้องการประกาศใช้งานตัวแปรสถานะภายใน Functional Component ผู้พัฒนาต้องเรียกใช้งาน React Hook ตัวใด?

ก. `useEffect`

ข. `useRef`

ค. `useState`

ง. `useMemo`

ข้อ 3: การประกาศรูปแบบฟังก์ชันสถานะตามหลักการทั่วไป เช่น `const [isOnline, setIsOnline] = useState(false);` ข้อใดกล่าวถึงบทบาทหน้าที่ของตัวแปร `setIsOnline` ได้ถูกต้อง?

ก. เป็นตัวแปรเก็บสถานะปัจจุบัน มีค่าประเภทข้อความ (String) เสมอ

ข. เป็นฟังก์ชันที่ทำหน้าที่เดียวในการลบตัวแปร `isOnline` ออกจากสเตตระบบ

ค. เป็นฟังก์ชันประเภท Setter ที่ใช้ในการอัปเดตและเปลี่ยนแปลงค่าของสถานะ `isOnline` เพื่อส่งสัญญาณจัดทำ Re-render หน้าจอ

ง. เป็นคำสั่งที่ส่งไปสร้างจุดกลมสีเขียวนีออนแสดงการเชื่อมต่อเน็ตเวิร์กของเครื่องอัตโนมัติ

ข้อ 4: ข้อใดคือความแตกต่างทางด้านสิทธิ์การแก้ไขค่า (Mutability) ระหว่าง State และ Props ของ React/React Native?

ก. State มีลักษณะใช้อ่านอย่างเดียว (Read-only) ส่วน Props สามารถเขียนโค้ดสลับเปลี่ยนค่าด้านในได้อย่างอิสระ

ข. State สามารถเปลี่ยนแปลงค่าได้ตามจังหวะพิมพ์สัมผัส ส่วน Props มีลักษณะใช้อ่านอย่างเดียวจากมุมมองของคอมโพเนนต์ลูกที่รับไป

ค. ทั้ง State และ Props ต่างไม่ยอมรับการเปลี่ยนแปลงค่าเลยเมื่อแอปพลิเคชันถูกบิลด์เสร็จ

ง. State ใช้แก้ไขเฉพาะประเภทข้อมูลตัวเลข ส่วน Props ใช้แก้ไขประเภทข้อความตัวอักษรเท่านั้น

ข้อ 5: แนวคิดแบบ Controlled Component ของช่องรับอินพุต `TextInput` มีลักษณะการจัดระเบียบข้อมูลอย่างไร?

ก. ปล่อยให้แป้นพิมพ์ของระบบควบคุมและจัดเก็บตัวหนังสือไว้ในฮาร์ดแวร์โดยตรง

ข. นำ State ของ React มาเป็นศูนย์กลางข้อมูลผูกติดกับคอมโพเนนต์ เพื่อคอยควบคุมและสะท้อนค่าตัวหนังสือปัจจุบันเข้าหากันตลอดเวลา

ค. ปิดแป้นคีย์บอร์ดทั้งหมดและรับข้อมูลด้วยวิธีลากผ่านแถบเสียงเท่านั้น

ง. ดึงสไตล์จากไฟล์ภายนอกมาควบคุมและตรวจสอบสีของหน้าแบบฟอร์ม

ข้อ 6: พร็อพส์ (Props) หลักคู่ใดของคอมโพเนนต์ `TextInput` ที่ต้องเขียนผูกทำงานร่วมกันเพื่อให้มีพฤติกรรมเป็น Controlled Component ที่สมบูรณ์?

ก. `placeholder` และ `keyboardType`

ข. `value` และ `onChangeText`

ค. `secureTextEntry` และ `autoCapitalize`

ง. `editable` และ `maxLength`

ข้อ 7: ในกรณีที่คอมโพเนนต์ลูกสองชิ้นที่ไม่มีความสัมพันธ์เป็นแม่-ลูกกันโดยตรง (เช่น ปุ่มสลับธีม และการ์ดแสดงสี) ต้องการใช้สถานะข้อมูลเดียวกัน วิธีการจัดสถาปัตยกรรมสเตตที่ดีที่สุดคืออะไร?

ก. เขียนคัดลอกไฟล์ State แบบเดียวกันไว้ในคอมโพเนนต์ลูกทั้งสองที่แยกอิสระจากกัน

ข. ประยุกต์ใช้เทคนิค "Lifting State Up" โดยย้าย State ขึ้นไปไว้ที่คอมโพเนนต์ที่เป็นบรรพบุรุษร่วมกันที่ใกล้ที่สุด (Common Parent)

ค. ยกเลิกการใช้ State และใช้รูปแบบ CSS Inline ในการเปลี่ยนสี

ง. ส่งข้อมูลผ่านทางเครือข่ายอินเทอร์เน็ตเพื่อเก็บข้อมูลที่ฐานข้อมูลกลางทุกรอบพิมพ์

ข้อ 8: คอมโพเนนต์ลูก (Child Component) สามารถส่งข้อมูลกลับขึ้นไปแจ้งเปลี่ยนแปลงหรืออัปเดตสเตตในคอมโพเนนต์แม่ (Parent Component) ได้อย่างไร?

ก. เรียกใช้ฟังก์ชัน `useState` ของแม่ตรง ๆ ผ่านการอ้างอิงชื่อคลาส

ข. แม่ต้องส่งฟังก์ชันประเภท Callback (ฟังก์ชันอัปเดตค่า) ลงไปให้ลูกในรูปแบบของ Props และให้ลูกเรียกใช้ฟังก์ชันนั้นพร้อมป้อนข้อมูลล่าสุดเป็นอาร์กิวเมนต์

ค. เขียนเปลี่ยนสเตตลงในไฟล์สไตล์ชีทภายนอกเพื่อให้คอมไพเลอร์เปลี่ยนค่า

ง. ไม่มีวิธีการตามข้อกำหนดของ React เนื่องจากข้อมูลไหลได้เฉพาะทิศทางลงล่างเท่านั้น

ข้อ 9: ข้อใดคือเป้าหมายหลักของการเขียนตรวจสอบความถูกต้องแบบฟอร์มในฝั่งไคลเอนต์ (Client-Side Form Validation)?

ก. เพื่อสแกนและเก็บฐานข้อมูลรหัสผ่านส่งต่อไปยังเซิร์ฟเวอร์แบบลับ ๆ

ข. เพื่อให้ผู้ใช้สัมผัสปุ่มกดและเห็นการตอบสนองได้ช้าลงเล็กน้อย

ค. เพื่อให้ข้อมูลที่ป้อนไม่ขัดต่อเงื่อนไขเบื้องต้น ส่งผลให้แสดงข้อเสนอแนะความผิดพลาด (Error Feedback) แก่ผู้ใช้ได้ทันที และช่วยลดการส่งรีเควสขยะไปยังระบบหลังบ้าน

ง. เพื่อเร่งความเร็วการส่งไฟล์รูปภาพขึ้นไคล์เซิร์ฟเวอร์

ข้อ 10: หากฟอร์มรับอินพุตยังกรอกข้อมูลไม่ถูกต้องหรือไม่ครบเงื่อนไข เราจะใช้เทคนิคใดในการปิดการทำงานของปุ่มตอบรับ `Pressable` เพื่อความปลอดภัย?

ก. ตั้งค่าพร็อพส์ `disabled={!isFormValid}` ร่วมกับการปรับสไตล์สีปุ่มให้จางลง (เช่น ใช้ Opacity ต่ำ) เมื่อข้อมูลไม่ถูกต้อง

ข. ตั้งค่าพร็อพส์ `editable={false}` เพื่อลบสุ่มปุ่มออกจากหน้าต่างจอหลัก

ค. เขียนฟังก์ชัน `Alert.alert()` ซ้อนในสไตล์เพื่อให้ผู้ใช้ลบตัวหนังสือทิ้ง

ง. บังคับเปลี่ยนคีย์บอร์ดของมือถือผู้ใช้ให้กลายเป็นหน้าปุ่มตัวเลขทั้งหมด

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: State ไม่ใช่ตัวควบคุมหรืออ่านสไตล์ภายนอกของระบบปฏิบัติการ

• ข. ถูก: State คือสภาพหน่วยความจำชั่วคราวที่เป็นส่วนตัวภายในคอมโพเนนต์ เมื่อเรียกตัวจัดการสเตตมาอัปเดตข้อมูล React จะจัดทำการ Re-render หน้าอินเทอร์เฟซเพื่อให้ผู้เรียนหรือผู้ใช้งานพบภาพสัดส่วนข้อมูลปัจจุบันของคอมโพเนนต์นั้นได้ทันที

• ค. และ ง. ผิด: Props เป็นคนละเรื่องกับสเตตภายใน และ State ไม่เกี่ยวข้องกับการจัดการพุชโนติฟาย

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `useEffect` ใช้จัดการผลกระทบข้างเคียง (Side Effects) เช่น การดึงค่า API หรือผูกเซนเซอร์เครื่อง

• ข. ผิด: `useRef` ใช้เก็บตัวแปรที่ใช้อ้างอิงโดยตรงหรืออ็อบเจกต์ที่เมื่อเปลี่ยนค่าแล้วจะไม่กระตุ้นการเรนเดอร์ใหม่

• ค. ถูก: `useState` เป็น Hook ดั้งเดิมที่ React เตรียมไว้ใช้สร้างและเข้าถึงตัวจัดการสเตตใน Functional Components

• ง. ผิด: `useMemo` ใช้แคชค่าการคำนวณที่ซับซ้อนเพื่อเพิ่มประสิทธิภาพ

เฉลย ข้อ 3: ตอบ ค.

• เหตุผลประกอบ:

- **ก. ผิด:** ตัวเก็บค่าคือ `isOnline` ไม่ใช่ตัวแปรฟังก์ชัน `setIsOnline`
- **ข. ผิด:** ฟังก์ชันใช้เปลี่ยนค่าสเตต ไม่ใช่คีย์ลบโครงสร้างตัวแปร
- **ค. ถูก:** ตัวแปรตัวที่สองที่ได้จากการทำ Array Destructuring ของ `useState` จะทำหน้าที่เป็นตัว Setter คอยรับเปลี่ยนสเตต และแจ้งสั่งการให้ตัวเอนจิ้นของ React เข้าสู่วงรอบการแสดงผลหน้าจอใหม่
- **ง. ผิด:** ไม่ได้เป็นคำสั่งออโต้สร้างจุดไอคอน แต่ผู้พัฒนาสามารถนำค่าของสเตต `isOnline` ไปคุมเงื่อนไขสีวงกลมได้เอง

เฉลย ข้อ 4: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ผิด:** สลับบทบาทกันอย่างสิ้นเชิง
  - **ข. ถูก:** State กำหนดในตนเองและเปลี่ยนข้อมูลได้เองเรื่อย ๆ ส่วน Props เป็นคุณสมบัติที่คนนอก (Component แม่) ยื่นส่งเข้ามาให้ โดยลูกที่รับไปห้ามแก้ไขสลับค่านั้นเองเพื่อไม่ให้กระแสข้อมูลขัดกัน
  - **ค. และ ง. ผิด:** มีการเปลี่ยนแปลงค่าได้ปกติยามรันไทม์ และทำงานได้ทุกประเภทข้อมูลอย่างไม่มีข้อจำกัดด้านชนิด

เฉลย ข้อ 5: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ผิด:** แป้นพิมพ์ไม่ควรเป็นผู้ตัดสินใจเก็บข้อความหลักเดี่ยว ๆ เพราะแอปจะไม่ทราบค่าปัจจุบันแบบทันที
  - **ข. ถูก:** Controlled Component หมายถึงการออกแบบให้อินพุตผูกข้อมูลข้อความปัจจุบันไว้บนสเตตของ React โดยตรง ทุกจังหวะการเปลี่ยนอักขระจะเข้าไปอัปเดตสเตต และตัวสเตตนั้นเองจะสะท้อนความจริงกลับมาป้อนบนหน้าต่างออินพุต
  - **ค. และ ง. ผิด:** ไม่ใช่การลบหรือควบคุมแป้น และไม่ใช่เรื่องการดีไซน์เฉดสีอย่างเดียว

เฉลย ข้อ 6: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ค. และ ง. ผิด:** พร็อพส์อื่น ๆ ช่วยอำนวยสไตล์และความปลอดภัยเบื้องต้น แต่ไม่เพียงพอต่อการควบคุมสถานะข้อมูล
  - **ข. ถูก:** เพื่อให้คอมโพเนนต์รับค่าอยู่ภายใต้การควบคุม (Controlled) พร็อพส์ `value` จะเป็นตัวชี้นำแสดงค่าสตริงของสเตต และพร็อพส์ `onChangeText` จะเป็นตัวส่งสัญญาณเมื่อผู้ใช้ป้อนอักขระล่าสุดเพื่อเปลี่ยนแปลงค่าสเตตตามลำดับ

เฉลย ข้อ 7: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การทำสำเนาสเตตแยกอิสระจะส่งผลให้หน้าจอทำงานไม่สอดคล้องกัน (เช่น ปุ่มเปลี่ยนธีมแล้วแต่การ์ดยังสีเดิม)

• ข. ถูก: ตามหลัก Lifting State Up การยก State ไปฝากไว้ที่บรรพบุรุษร่วม (Parent Component) จะช่วยให้เราแชร์สเตตชิ้นนั้นส่งผ่านลงมาในรูปแบบ Props ให้ลูกทั้งสองคนได้พร้อม ๆ กันอย่างเป็นระบบและสม่ำเสมอ

• ค. และ ง. ผิด: การใช้ CSS ตรงไม่แก้ไขการเปลี่ยนข้อมูล และการโหลดเน็ตเวิร์กทุกรอบพิมพ์จะทำให้เครื่องช้าและสิ้นเปลือง

เฉลย ข้อ 8: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: React ไม่อนุญาตให้ดึงใช้งานอ้างอิงข้ามฟังก์ชันสเตตของตัวแม่ตรง ๆ ด้วยความสามารถทั่วไป

• ข. ถูก: เราจำเป็นต้องกำหนดพร็อพส์ให้รับฟังก์ชัน Callback จากคอมโพเนนต์แม่ เมื่อเกิดเหตุสัมผัสปุ่มหรืออินพุต คอมโพเนนต์ลูกจะกระตุ้นฟังก์ชันนั้นย้อนกลับไป และสั่งเปลี่ยนค่าสเตตต้นฉบับส่งผลให้แม่และลูกเรนเดอร์อัปเดตกันถ้วนหน้า

• ค. และ ง. ผิด: ไฟล์สไตล์ชีทไม่มีอิมแพคควบคุมข้อมูลสเตต และสามารถส่งข้อมูลขึ้นด้านบนได้ผ่านรูปแบบการใช้ Callback Function

เฉลย ข้อ 9: ตอบ ค.

• เหตุผลประกอบ:

• ก. และ ข. ผิด: ไม่เกี่ยวกับสปายแวร์หลังบ้าน หรือการดึงจังหวะสัมผัสให้ตอบสนองช้าลง

• ค. ถูก: การทำ Form Validation ฝั่งมือถือปลายทางช่วยเตือนคำแนะนำแก่ผู้ใช้งานแอปอย่างฉับพลันเมื่อป้อนข้อความผิดพลาด และป้องกันการส่งขยะ (ข้อมูลไม่ครบหรือผิดโครงสร้าง) ไปเซิร์ฟเวอร์

• ง. ผิด: ระบบส่งรูปขึ้นคลาวด์จะดีขึ้นตามขนาดไฟล์และแบนด์วิดท์ความเร็วเน็ต ไม่เกี่ยวกับกฎข้อบังคับของฟอร์มอินพุต

เฉลย ข้อ 10: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: คอมโพเนนต์ `Pressable` และคอมโพเนนต์ปุ่มอื่น ๆ มีพร็อพส์ `disabled` ซึ่งจะระงับเหตุการณ์ทริกเกอร์ `onPress` เมื่อค่าเป็น `true` ผู้พัฒนาสามารถนำการประเมินสเตตฟอร์มมาผูกร่วมและเขียนสไตล์ปรับเปลี่ยนสปอตไลท์ความสว่างปุ่มได้ชัดเจน

• **ข. ผิด:** `editable` เป็นคุณสมบัติสำหรับกล่อง `TextInput` ไม่ใช่ปุ่มกด

• **ค. และ ง. ผิด:** `Alert.alert` ไม่ระงับการสัมผัสปุ่ม และประเภทคีย์บอร์ดตัวเลขใช้ควบคุมอินพุตเท่านั้น

ปฏิบัติการท้ายบทเรียน: การพัฒนาแบบฟอร์มประเมินความพึงพอใจพร้อมระบบแสดงผลเรียลไทม์

ปฏิบัติการนี้จะฝึกฝนทักษะการประยุกต์ใช้สถานะ (State) การเชื่อมโยงคอมโพเนนต์ลูกด้วยหลักการ Lifting State Up และการเขียนระบบฟอร์มลงทะเบียนประเมินคุณภาพสินค้าพร้อมตรวจเช็คเงื่อนไขความถูกต้อง (Form Validation) โดยจำลองหน้าจอ **ระบบประเมินความพึงพอใจของลูกค้า (Customer Feedback & Review Form)**

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้งาน `useState` Hook ในการควบคุมการทำงานของแบบฟอร์มที่มีหลายช่องกรอกข้อมูล (Multi-input Form)

2. สามารถเขียนฟังก์ชันตรวจสอบความถูกต้องของข้อมูลแบบเรียลไทม์ (Live Form Validation) และดักจับความยาวข้อความได้

3. สามารถส่งผ่านสเตตอัปเดตย้อนกลับจากชิ้นส่วนคอมโพเนนต์ย่อยแบบปุ่มตัวเลือกผ่านกลไก Lifting State Up

4. สามารถเชื่อมโยงข้อมูลสเตตไปแสดงผลบนการ์ดพรีวิวจำลอง (Live Card Preview) ได้อย่างสม่ำเสมอ

5. สามารถจำกัดการโต้ตอบของปุ่มสัมผัสด้วยพร็อพส์ `disabled` และปรับสไตล์อย่างสอดคล้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องเขียนโค้ดและโครงสร้างแอปประเมินความพึงพอใจลงในโฟลเดอร์ของตนเอง โดยมีองค์ประกอบการทำงานดังนี้:

รายละเอียดเงื่อนไขและเลย์เอาต์แอป:

1. **ส่วนบน: การ์ดตัวอย่างแสดงผลการรีวิว (Live Review Card Preview)**

• แสดงข้อความประเภทชื่อผู้รีวิว คะแนนดาวจำลอง (1-5 ดาว) และข้อเสนอแนะความพึงพอใจ

• ข้อมูลบนการ์ดใบนี้จะเปลี่ยนแปลงสดทันทีเมื่อผู้เรียนพิมพ์ข้อความ หรือเลือกคะแนนดาวในกลุ่มแบบฟอร์ม

2. **ส่วนกลาง: แบบฟอร์มประเมิน (Feedback Form Container)**

• **กล่องกรอกชื่อผู้รีวิว:** ตัวหนังสือความยาวขั้นต่ำ 2 ตัวอักษร หากไม่ถึงเกณฑ์ให้แสดงกรอบสีส้มแดงเตือน และปิดระบบเดาภาษาอังกฤษอัตโนมัติ

• **ส่วนเลือกคะแนนประเมิน (Star Rating Selector):**

• เป็นคอมโพเนนต์ลูกย่อยชื่อ `components/RatingSelector.tsx`

• มีปุ่มคะแนน 1 ถึง 5 เรียงกันแนวนอน

• ปุ่มที่ถูกเลือกจะมีสีพื้นหลังสว่างเข้มกว่าปุ่มอื่น และเมื่อกดเลือกคะแนนใด ๆ จะต้องใช้หลักการ Lifting State Up เพื่อส่งสัญญาณตัวเลขคะแนนกลับไปบันทึกที่ State หลักในคอมโพเนนต์แม่

• **กล่องคำแนะนำ (Comment Textarea):** รับคำบรรยายความพึงพอใจขั้นต่ำ 10 ตัวอักษร หากไม่ถึงเกณฑ์ให้แสดงข้อความคำแนะนำสีแดงด้านล่าง

3. **ส่วนล่าง: ปุ่มกดส่งข้อมูลประเมิน (Submit & Reset Layout)**

• **ปุ่มกดส่ง (Submit):** ล็อกไม่ให้คลิกปุ่มสัมผัสทำงานได้หากกรอกข้อมูลชื่อไม่ครบ 2 อักษร ข้อคิดเห็นไม่ถึง 10 อักษร หรือยังไม่ได้เลือกกดคะแนนประเมิน

• **ปุ่มล้างค่า (Reset):** กดสัมผัสเพื่อคืนสถานะสเตตฟอร์มทั้งหมดกลับเป็นค่าเริ่มต้นเดิม

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: สร้างชิ้นส่วนเลือกระดับคะแนนด้วยเทคนิค Lifting State Up

สร้างไฟล์คอมโพเนนต์ใหม่ชื่อ `components/RatingSelector.tsx` เพื่อเป็นชุดปุ่มเลือกคะแนน 1-5 โดยให้ลูกส่งค่าตัวเลขที่เลือกลับคืนไปให้แม่ผ่าน Props ฟังก์ชัน:

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

// ข้อกำหนด Props: รับระดับคะแนนปัจจุบัน และฟังก์ชัน Callback ส่งคะแนนใหม่กลับขึ้นไป
interface RatingSelectorProps {
  selectedRating: number;
  onSelectRating: (rating: number) => void;
}

export default function RatingSelector({ selectedRating, onSelectRating }:
RatingSelectorProps) {
  const ratings = [1, 2, 3, 4, 5];

  return (
```

```
    <View style={styles.container}>
      <Text style={styles.label}>เลือกคะแนนความพึงพอใจ (1 - 5 ดาว)</Text>
      <View style={styles.starRow}>
        {ratings.map((num) => {
          const isSelected = num === selectedRating;
          return (
            <Pressable
              key={num}
              onPress={() => onSelectRating(num)} // ทริกเกอร์ฟังก์ชันส่งค่ากลับขึ้นด้านบน
              style={[
                styles.starButton,
                isSelected ? styles.starSelected : styles.starUnselected
      ]}
            >
              <Text style={[styles.starText, isSelected ?
styles.starTextActive : null]}>
        ☆ {num}
              </Text>
            </Pressable>
    );
        })}
      </View>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    marginVertical: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 8,
  },
  starRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    gap: 8,
  },
  starButton: {
    flex: 1,
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
    borderWidth: 1,
  },
  starSelected: {
```

```
    backgroundColor: '#f59e0b',
    borderColor: '#d97706',
  },
  starUnselected: {
    backgroundColor: '#0f172a',
    borderColor: '#334155',
  },
  starText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  starTextActive: {
    color: '#ffffff',
  },
});
```

ขั้นตอนที่ 2: ผูกข้อมูล สเตต และการแสดงตัวอย่างการ์ด ในหน้าจอหลัก app/index.tsx

เขียนแบบฟอร์มพร้อมเงื่อนไขการตรวจสอบคำเตือน UI ที่เปลี่ยนไปตามสเตตแบบเรียลไทม์:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import RatingSelector from '../components/RatingSelector'; // โหลดคอมโพเนนต์เลือกดาว

export default function App() {
 // สเตตเก็บข้อมูลช่องป้อนแบบฟอร์ม
  const [name, setName] = useState<string>('');
  const [rating, setRating] = useState<number>(0);
  const [comment, setComment] = useState<string>('');

 // สเตตเก็บแจ้งเตือนคำเตือน
  const [nameTouched, setNameTouched] = useState<boolean>(false);
  const [commentTouched, setCommentTouched] = useState<boolean>(false);

 // คำนวณความถูกต้องแบบเรียลไทม์
```

```
  const isNameInvalid = nameTouched && name.trim().length < 2;
  const isCommentInvalid = commentTouched && comment.trim().length < 10;

 // ตรวจสอบว่าปุ่มส่งข้อมูลควรถูกเปิดความต้องการใช้งานหรือไม่
  const isFormValid = name.trim().length >= 2 && comment.trim().length >=
10 && rating > 0;

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert(
        'ขอบคุณสำหรับการประเมิน',
        `ชื่อผู้ประเมิน: ${name}\nคะแนนความพอใจ: ${rating}/5 ดาว\nความคิดเห็น: ${comment}`
   );
    }
  };

  const handleReset = () => {
    setName('');
    setRating(0);
    setComment('');
    setNameTouched(false);
    setCommentTouched(false);
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <Text style={styles.headerTitle}>ส่งแบบประเมินบริการ</Text>

        {/* 1. Live Card Preview Area */}
        <View style={styles.previewCard}>
          <Text style={styles.previewTitle}>แสดงผลตัวอย่างคำวิจารณ์ (Live
Preview)</Text>
          <View style={styles.previewBody}>
            <Text style={styles.previewNameText}>
        👤 {name.trim() !== '' ? name : 'ยังไม่ได้ระบุชื่อผู้ประเมิน'}
            </Text>
            <Text style={styles.previewStarsText}>
              {rating > 0 ? '★'.repeat(rating) + '☆'.repeat(5 - rating) :
'☆☆☆☆☆ (กรุณาเลือกคะแนน)'}
            </Text>
            <Text style={styles.previewCommentText}>
        📝 {comment.trim() !== '' ? comment : 'พิมพ์ความคิดเห็นของคุณที่ฟอร์มด้านล่าง...'}
            </Text>
          </View>
        </View>
```

```
        {/* 2. Feedback Form Input Control */}
        <View style={styles.formCard}>

          {/* ช่องป้อนชื่อผู้รีวิว */}
          <View style={styles.inputGroup}>
            <Text style={styles.label}>ชื่อและนามสกุลลูกค้า</Text>
            <TextInput
              style={[styles.input, isNameInvalid ? styles.inputInvalid :
null]}
              placeholder="กรอกชื่อของคุณอย่างน้อย 2 ตัวอักษร"
              placeholderTextColor="#64748b"
              value={name}
              onChangeText={(text) => {
                setName(text);
                setNameTouched(true);
              }}
              autoCapitalize="none"
              autoCorrect={false}
      />
            {isNameInvalid && (
              <Text style={styles.errorText}>ข้อบกพร่อง: ชื่อต้องการอักขระอย่างน้อย 2 ตัวขึ้นไป
</Text>
      )}
          </View>

          {/* เรียกใช้งาน Custom Selector ผ่าน Props ด้วยกลไก Lifting State Up */}
          <RatingSelector selectedRating={rating}
onSelectRating={setRating} />

        {/* ช่องกรอกข้อคิดเห็นประเมินผล */}
          <View style={styles.inputGroup}>
            <Text style={styles.label}>รายละเอียดความพึงพอใจการใช้บริการ</Text>
            <TextInput
              style={[styles.input, styles.textArea, isCommentInvalid ?
styles.inputInvalid : null]}
              placeholder="บรรยายสิ่งที่คุณอยากแนะนำอย่างน้อย 10 ตัวอักษร..."
              placeholderTextColor="#64748b"
              value={comment}
              onChangeText={(text) => {
                setComment(text);
                setCommentTouched(true);
              }}
              multiline={true}
              numberOfLines={4}
      />
            {isCommentInvalid && (
              <Text style={styles.errorText}>ข้อบกพร่อง: กรุณากรอกรายละเอียดอย่างน้อย 10
ตัวอักษร</Text>
      )}
```

```
          </View>

          {/* ปุ่มจัดรายการ Submit และ Reset */}
          <View style={styles.buttonRow}>
            <Pressable
              onPress={handleReset}
              style={({ pressed }) => [
              styles.resetButton,
                pressed ? styles.resetButtonPressed : null
        ]}
            >
              <Text style={styles.resetButtonText}>ล้างฟอร์ม</Text>
            </Pressable>

            <Pressable
              disabled={!isFormValid}
              onPress={handleSubmit}
              style={({ pressed }) => [
                styles.submitButton,
         !isFormValid ? styles.submitButtonDisabled : null,
                pressed && isFormValid ? styles.submitButtonPressed : null
        ]}
            >
              <Text style={styles.submitButtonText}>ส่งข้อมูลรีวิว</Text>
            </Pressable>
          </View>

        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  // รายละเอียด CSS Style จะรวมอยู่ในโครงสร้างเฉลยในส่วนถัดไป
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถเปิดดูซอร์สโค้ดที่ถูกต้องรวมการสไตล์ชีทครบถ้วน โดยคัดลอกไฟล์ต่อไปนี้ไปทดสอบการทำงานบนตัวเครื่อง:

4.1 ซอร์สโค้ดไฟล์ปุ่มคะแนน: **components/RatingSelector.tsx**

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

interface RatingSelectorProps {
  selectedRating: number;
```

```
  onSelectRating: (rating: number) => void;
}

export default function RatingSelector({ selectedRating, onSelectRating }:
RatingSelectorProps) {
  const ratings = [1, 2, 3, 4, 5];

  return (
    <View style={styles.container}>
      <Text style={styles.label}>เลือกคะแนนความพึงพอใจ (1 - 5 ดาว)</Text>
      <View style={styles.starRow}>
        {ratings.map((num) => {
          const isSelected = num === selectedRating;
          return (
            <Pressable
              key={num}
              onPress={() => onSelectRating(num)}
              style={[
                styles.starButton,
                isSelected ? styles.starSelected : styles.starUnselected
      ]}
            >
              <Text style={[styles.starText, isSelected ?
styles.starTextActive : null]}>
        ☆ {num}
              </Text>
            </Pressable>
    );
        })}
      </View>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    marginVertical: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 8,
  },
  starRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    gap: 8,
  },
  starButton: {
```

```
    flex: 1,
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
    borderWidth: 1,
  },
  starSelected: {
    backgroundColor: '#f59e0b',
    borderColor: '#d97706',
  },
  starUnselected: {
    backgroundColor: '#0f172a',
    borderColor: '#334155',
  },
  starText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  starTextActive: {
    color: '#ffffff',
  },
});
```

4.2 ซอร์สโค้ดหน้าหลักฟอร์มโต้ตอบ: **app/index.tsx**

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import RatingSelector from '../components/RatingSelector';

export default function App() {
  const [name, setName] = useState<string>('');
  const [rating, setRating] = useState<number>(0);
  const [comment, setComment] = useState<string>('');

  const [nameTouched, setNameTouched] = useState<boolean>(false);
  const [commentTouched, setCommentTouched] = useState<boolean>(false);

  const isNameInvalid = nameTouched && name.trim().length < 2;
```

```
  const isCommentInvalid = commentTouched && comment.trim().length < 10;

  const isFormValid = name.trim().length >= 2 && comment.trim().length >= 10
&& rating > 0;

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert(
        'ส่งข้อมูลรีวิวสำเร็จ',
        `ขอบคุณสำหรับข้อแนะนำที่มีค่า!\n\nผู้รีวิว: ${name}\nคะแนน: ${rating}/5 ดาว\nเนื้อหา:
${comment}`
      );
      handleReset();
    }
  };

  const handleReset = () => {
    setName('');
    setRating(0);
    setComment('');
    setNameTouched(false);
    setCommentTouched(false);
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <Text style={styles.headerTitle}>แบบประเมินการให้บริการ</Text>

        {/* 1. Live Preview Card */}
        <View style={styles.previewCard}>
          <Text style={styles.previewTitle}>ตัวอย่างรีวิวสด (Live Preview)</Text>
          <View style={styles.previewBody}>
            <Text style={styles.previewNameText}>
              {name.trim() !== '' ? name : 'ยังไม่ได้ป้อนชื่อของคุณ'}
            </Text>
            <Text style={styles.previewStarsText}>
              {rating > 0
                ? '★'.repeat(rating) + '☆'.repeat(5 - rating)
                : '☆☆☆☆☆ (กรุณาเลือกคะแนนให้คะแนนดาว)'}
            </Text>
            <Text style={styles.previewCommentText}>
              {comment.trim() !== '' ? comment : 'รายละเอียดความคิดเห็นของคุณจะปรากฏที่นี่...'}
            </Text>
          </View>
        </View>

        {/* 2. Interaction Inputs Form */}
```

```
        <View style={styles.formCard}>

          <View style={styles.inputGroup}>
            <Text style={styles.label}>ชื่อ-นามสกุลผู้รีวิว</Text>
            <TextInput
              style={[styles.input, isNameInvalid ? styles.inputInvalid :
null]}
              placeholder="กรอกชื่อ-สกุลจริงของคุณ"
              placeholderTextColor="#64748b"
              value={name}
              onChangeText={(text) => {
                setName(text);
                setNameTouched(true);
              }}
              autoCapitalize="none"
              autoCorrect={false}
      />
            {isNameInvalid && (
              <Text style={styles.errorText}>* กรุณากรอกชื่อจริงอย่างน้อย 2 ตัวอักษร
</Text>
      )}
          </View>

          <RatingSelector selectedRating={rating}
onSelectRating={setRating} />

          <View style={styles.inputGroup}>
            <Text style={styles.label}>ข้อแนะนำและระดับข้อเสนอแนะ</Text>
            <TextInput
              style={[styles.input, styles.textArea, isCommentInvalid ?
styles.inputInvalid : null]}
              placeholder="บอกเหตุผลหรือสิ่งที่ระบบควรปรับปรุง (อย่างน้อย 10 ตัวอักษร)..."
              placeholderTextColor="#64748b"
              value={comment}
              onChangeText={(text) => {
                setComment(text);
                setCommentTouched(true);
          }}
              multiline={true}
              numberOfLines={4}
      />
            {isCommentInvalid && (
              <Text style={styles.errorText}>* ข้อความยาวไม่เพียงพอ (ต้องการขั้นต่ำ 10
อักษร)</Text>
      )}
          </View>

          <View style={styles.buttonRow}>
            <Pressable
              onPress={handleReset}
```

```
              style={({ pressed }) => [
                styles.resetButton,
                pressed ? styles.resetButtonPressed : null
            ]}
              >
                <Text style={styles.resetButtonText}>ล้างแบบฟอร์ม</Text>
              </Pressable>

              <Pressable
                disabled={!isFormValid}
                onPress={handleSubmit}
                style={({ pressed }) => [
               styles.submitButton,
          !isFormValid ? styles.submitButtonDisabled : null,
                pressed && isFormValid ? styles.submitButtonPressed : null
            ]}
              >
                <Text style={styles.submitButtonText}>บันทึกความคิดเห็น</Text>
              </Pressable>
            </View>

          </View>
        </ScrollView>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
    paddingBottom: 40,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 16,
  },
  previewCard: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    borderRadius: 16,
    padding: 16,
```

```
    marginBottom: 20,
  },
  previewTitle: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#38bdf8',
    textTransform: 'uppercase',
    marginBottom: 10,
    letterSpacing: 0.5,
  },
  previewBody: {
    backgroundColor: '#0f172a',
    padding: 14,
    borderRadius: 8,
    gap: 8,
  },
  previewNameText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  previewStarsText: {
    color: '#fbbf24',
    fontSize: 18,
  letterSpacing: 2,
  },
  previewCommentText: {
    color: '#cbd5e1',
    fontSize: 14,
    fontStyle: 'italic',
    lineHeight: 20,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderColor: '#334155',
    borderWidth: 1,
  },
  inputGroup: {
    marginBottom: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 6,
```

```
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 14,
  },
  inputInvalid: {
    borderColor: '#f43f5e',
  },
  textArea: {
    height: 90,
    paddingTop: 10,
    textAlignVertical: 'top',
  },
  errorText: {
    color: '#f43f5e',
    fontSize: 12,
    marginTop: 4,
  },
  buttonRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 16,
    gap: 12,
  },
  resetButton: {
    flex: 1,
    height: 48,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    backgroundColor: '#1e293b',
    justifyContent: 'center',
    alignItems: 'center',
  },
  resetButtonPressed: {
    backgroundColor: '#334155',
  },
  resetButtonText: {
    color: '#cbd5e1',
    fontSize: 15,
    fontWeight: 'bold',
```

```
  },
  submitButton: {
    flex: 2,
    height: 48,
    borderRadius: 8,
    backgroundColor: '#3b82f6',
    justifyContent: 'center',
    alignItems: 'center',
  },
  submitButtonDisabled: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    opacity: 0.4,
  },
  submitButtonPressed: {
    backgroundColor: '#2563eb',
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 15,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

React. (2024). *State: A Component's Memory*. จาก https://react.dev/learn/state-a-components-memory

React. (2024). *Synchronizing with Effects*. จาก https://react.dev/learn/synchronizing-with-effects

React Native. (2024). *Handling Text Input*. จาก https://reactnative.dev/docs/handling-text-input